BABU BANARASI DAS UNIVERSITY

LUCKNOW

Session : 2024-2025

SCHOOL OF COMPUTER APPLICATION

PROJECT

Mongo DB

Project Name :- MongoDB Insights Finding

Submitted By

Javed Husain

Submitted To

Mr. Ankit Soni

Batch :- BCADS23

PROJECT

1. Complex Filters & Projections

Q1. List the names and departments of students who have more than 85% attendance and are skilled in both "MongoDB" and "Python".

Solution :

```
db.students.find(
  { attendance: { $gt: 85 }, skills: { $all: ["MongoDB", "Python"] } },
  { _id: 0, name: 1, department: 1, attendance: 1, skills: 1 }
)
```

Output :

```
mydatabase> db.students.find( // Name: Javed Husain Registration No: 1240258206
...     { attendance: { $gt: 85 }, skills: { $all: ["MongoDB", "Python"] } },
...     { _id: 0, name: 1, department: 1, attendance: 1, skills: 1 }
... );

mydatabase>
```

Conclusion : The db.students.find() query you provided is designed to locate students who meet two specific criteria: they have an attendance score **greater than 85**, and their skills array includes **both "MongoDB" and "Python"**.

Q2. Show all faculty who are teaching more than 2 courses. Display their names and the total number of courses they teach.

Solution :

```
db.faculty.aggregate([
  { $project: { name: 1, totalCourses: { $size: "$courses" } } },
  { $match: { totalCourses: { $gt: 2 } } }
]);
```

Output :

```
mydatabase> // Name: Javed Husain Registration No: 1240258206
... db.faculty.aggregate([
...   { $project: { name: 1, totalCourses: { $size: "$courses" } } },
...   { $match: { totalCourses: { $gt: 2 } } }
... ]);
[
  { _id: 'F029', name: 'Charles Newton', totalCourses: 3 },
  { _id: 'F032', name: 'Julia Cole', totalCourses: 3 },
  { _id: 'F040', name: 'Darrell Velasquez', totalCourses: 3 },
  { _id: 'F048', name: 'Michael Poole', totalCourses: 3 },
  { _id: 'F051', name: 'John Duran', totalCourses: 3 },
  { _id: 'F061', name: 'Daniel Allen', totalCourses: 3 },
  { _id: 'F083', name: 'Matthew Hanna', totalCourses: 3 },
  { _id: 'F084', name: 'Michael Johnson', totalCourses: 3 },
  { _id: 'F100', name: 'Robert Lara', totalCourses: 3 }
]
mydatabase>
```

Conclusion : This aggregation query identifies faculty members who teach more than two courses. The query returns the names of these faculty members and the total number of courses they teach

2. Joins ($lookup) and Aggregations

Q3. Write a query to show each student's name along with the course titles they are enrolled in (use $lookup between enrollments, students, and courses).

Solution :

```
db.enrollments.aggregate([
  { $lookup: { from: "students", localField: "student_id", foreignField: "_id", as: "studentInfo" }
  },
  { $lookup: { from: "courses", localField: "course_id", foreignField: "_id", as: "courseInfo" } },
  { $project: {
      _id: 0,
      studentName: { $arrayElemAt: ["$studentInfo.name", 0] },
      courseTitles: "$courseInfo.title"
    }
  }
]);
```

Output :

```
mydatabase> db.enrollments.aggregate(      // Name: Javed Husain Registration No: 1240258206
...     [
...         {
...             $lookup: {
...                 from: "students",
...                 localField: "student_id",
...                 foreignField: "_id",
...                 as: "studentInfo"
...             }
...         },
...         {
...             $lookup: {
...                 from: "courses",
...                 localField: "course_id",
...                 foreignField: "_id",
...                 as: "courseInfo"
...             }
...         },
...         {
...             $project: {
...                 _id: 0,
...                 studentName: { $arrayElemAt: ["$studentInfo.name", 0] },
...                 courseTitles: "$courseInfo.title"
...             }
...         }
...     ]
... );
[
  {
    studentName: 'Alexandra Bailey',
    courseTitles: [ 'Reactive neutral adapter' ]
  },
  {
    studentName: 'Megan Taylor',
    courseTitles: [ 'Sharable bifurcated paradigm' ]
  },
  {
    studentName: 'Alejandro Hart',
    courseTitles: [ 'Focused user-facing paradigm' ]
  },
  {
    studentName: 'Timothy Sparks',
    courseTitles: [ 'Focused user-facing paradigm' ]
  },
```

Conclusion :

This aggregation query uses two $lookup stages to join the enrollments collection with the students and courses collections. The final $project stage then selects and formats the output to display each student's name along with the titles of the courses they are enrolled in.

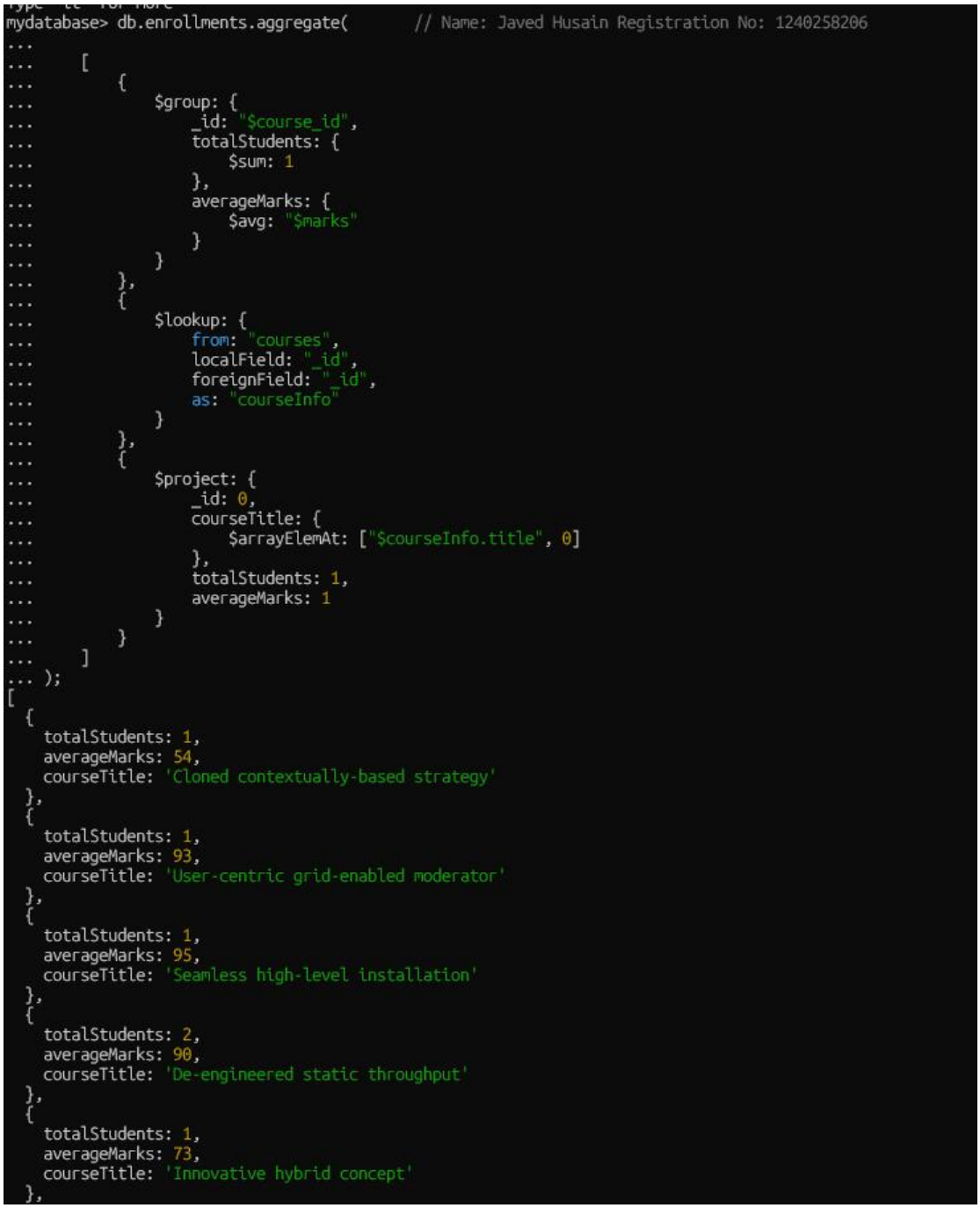
Q4. For each course, display the course title, number of students enrolled, and average marks
(use $group).

Solution :

```
db.enrollments.aggregate(      // Name: Javed Husain Registration No: 1240258206

  [
    {
      $group: {
        _id: "$course_id",
        totalStudents: {
          $sum: 1
        },
        averageMarks: {
          $avg: "$marks"
        }
      }
    },
    {
      $lookup: {
        from: "courses",
        localField: "_id",
        foreignField: "_id",
        as: "courseInfo"
      }
    },

    {

      $project: {
        _id: 0,
        courseTitle: {
          $arrayElemAt: ["$courseInfo.title", 0]
        },

        totalStudents: 1,
        averageMarks: 1

      }

    }

  ]
```

);

Output :



Conclusion :

This aggregation query processes data from the enrollments collection to provide a summary of course performance. It first groups students by course_id to calculate the total number of students and the average marks for each course. It then joins this aggregated data with the courses collection to display the course title alongside the calculated metrics.

3. Grouping, Sorting, and Limiting

Q5. Find the top 3 students with the highest average marks across all enrolled courses.

Solution :

```
db.enrollments.aggregate(   // Name: Javed Husain Registration No: 1240258206
  [

    {
      $group: {
        _id: "$student_id",
        averageMarks: { $avg: "$marks" }
      }
    },

    {
      $sort: { averageMarks: -1}
    },
    {
      $limit: 3
    },

    {
      $lookup: {
        from: "students",
        localField: "_id",
        foreignField: "_id",
        as: "studentInfo"
      }

    },

    {
      $project: {
        _id: 0,
        studentName: { $arrayElemAt: ["$studentInfo.name", 0]},
        averageMarks: 1
      }

    }

  ]

);
```

Output :

```
mydatabase>
... db.enrollments.aggregate(   // Name: Javed Husain Registration No: 1240258206
...     [
...         {
...             $group: {
...                 _id: "$student_id",
...                 averageMarks: { $avg: "$marks" }
...             }
...         },
...         {
...             $sort: { averageMarks: -1}
...         },
...         {
...             $limit: 3
...         },
...         {
...             $lookup: {
...                 from: "students",
...                 localField: "_id",
...                 foreignField: "_id",
...                 as: "studentInfo"
...             }
...         },
...         {
...             $project: {
...                 _id: 0,
...                 studentName: { $arrayElemAt: ["$studentInfo.name", 0]},
...                 averageMarks: 1
...             }
...         }
...     ]
... );
[
  { averageMarks: 100, studentName: 'Diane Phillips' },
  { averageMarks: 98, studentName: 'Brandon Rios' },
  { averageMarks: 94, studentName: 'Christopher Benson' }
]
```

Conclusion :

This aggregation query identifies the **top 3 students with the highest average marks** across all courses they are enrolled in. It achieves this by first grouping the enrollment records by student to calculate the average marks, then sorts the results in descending order, and finally limits the output to the top three students. The $lookup and $project stages are used to join the student information and display their names and average marks in a clean, readable format.

Q6. Count how many students are in each department. Display the department with the highest number of students.

Solution :

```
db.students.aggregate(   // Name: Javed Husain Registration No: 1240258206
  [
    {
      $group: {
        _id: "$department",
        totalStudents: { $sum: 1}
      }
    },
    {
      $sort: { totalStudents: -1}
    },
    {
      $limit: 1
    }, { $project: {_id: 0,
        department: "$_id",
        totalStudents: 1
      }
    }
  ]
);
```

Output :

```
mydatabase>
... db.students.aggregate(   // Name: Javed Husain Registration No: 1240258206
...
...     [
...         {
...             $group: {
...                 _id: "$department",
...                 totalStudents: { $sum: 1}
...             }
...         },
...         {
...             $sort: { totalStudents: -1}
...         },
...         {
...             $limit: 1
...         },
...         {
...             $project: {
...                 _id: 0,
...                 department: "$_id",
...                 totalStudents: 1
...             }
...         }
...     ]
... );
[ { totalStudents: 23, department: 'Electrical' } ]
mydatabase>
```

Conclusion : This MongoDB aggregation query finds the department with the highest number of students by first counting students per department, then sorting the results to find the highest count, and finally displaying only that single department.

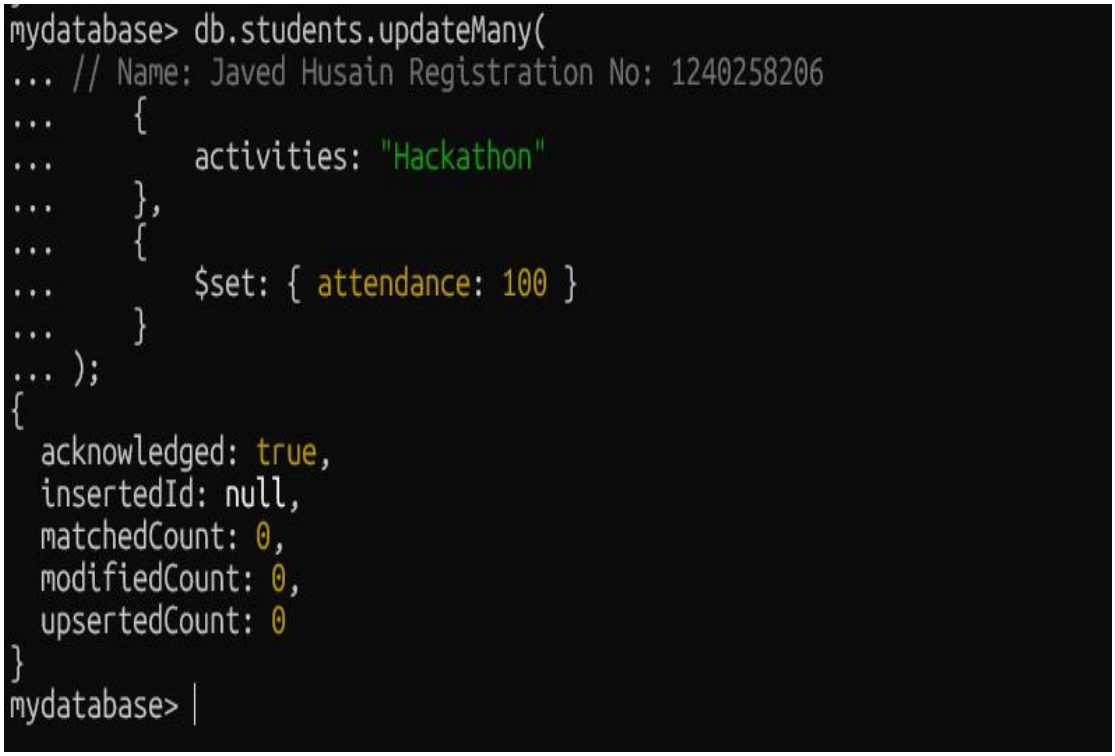
4. Update, Upsert, and Delete

Q7. Update attendance to 100% for all students who won any "**Hackathon**".

Solution :

```
db.students.updateMany(
        // Name: Javed Husain Registration No: 1240258206
  {
    activities: "Hackathon"
  },
  {
    $set: { attendance: 100 }
  }
);
```

Output :



Conclusion :

This MongoDB query uses the updateMany() method to update multiple student records at once. It identifies all students who have "Hackathon" in their activities array and sets their attendance field to 100%. This is an efficient way to apply a change to multiple documents that share a common characteristic.

Q8. Delete all student activity records where the activity year is before 2022

Solution :

```
db.activities.deleteMany(
        // Name: Javed Husain Registration No: 1240258206
  {
    year: { $lt: 2022 }
  }
);
```

Output :

```
mydatabase> db.activities.deleteMany(
...     // Name: Javed Husain Registration No: 1240258206
...     {
...         year: { $lt: 2022 }
...     }
... );
{ acknowledged: true, deletedCount: 0 }
mydatabase>
```

Conclusion : This query uses deleteMany() to efficiently remove all documents from the activities collection where the year field is a value less than 2022.

Q9. Upsert a course record for "**Data Structures**" with ID "**C150**" and credits 4—if it doesn't exist, insert it; otherwise update its title to "**Advanced Data Structures**".

Solution :

```
db.courses.updateOne(
  {_id: "C150"},        // Name: Javed Husain Registration No: 1240258206

  {
    $set: { title: "Advanced Data Structures", credits: 4}
  },
  { upsert: true }
);
```

Output :

```
mydatabase> db.courses.updateOne(
...     {_id: "C150"},          // Name: Javed Husain Registration No: 1240258206
...
...     {
...         $set: { title: "Advanced Data Structures", credits: 4}
...     },
...     { upsert: true }
... );
{
  acknowledged: true,
  insertedId: 'C150',
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
mydatabase>
```

Conclusion : This MongoDB query uses updateOne() with the upsert: true option to either create a new course record or update an existing one. It targets the document with _id: "C150" and ensures its title is "Advanced Data Structures" and its credits are 4.

5. Array & Operator Usage

Q10. Find all students who have "**Python**" as a skill but not "**C++**".

Solution :

```
db.students.find(
  {              // Name: Javed Husain Registration No: 1240258206
    $and: [
      { skills: "Python" },
      { skills: { $ne: "C++" } }
    ]
  },
  {
    _id: 0,
    name: 1,
    skills: 1
  }
);
```

Output :

```
mydatabase> db.students.find(
...     {              // Name: Javed Husain Registration No: 1240258206
...         $and: [
...             { skills: "Python" },
...             { skills: { $ne: "C++" } }
...         ]
...     },
...     {
...         _id: 0,
...         name: 1,
...         skills: 1
...     }
... );
[
  { name: 'Kyle Hale', skills: [ 'Python', 'Java' ] },
  { name: 'Cody Whitehead', skills: [ 'JavaScript', 'Python' ] },
  { name: 'Thomas Jackson', skills: [ 'Python', 'AutoCAD' ] },
  { name: 'Steven Wong', skills: [ 'MongoDB', 'Python' ] },
  { name: 'Cheryl Jackson', skills: [ 'Research', 'Python' ] },
  { name: 'Mr. Darius Newman', skills: [ 'Python', 'SQL' ] },
  { name: 'Derrick Humphrey', skills: [ 'Python', 'Java' ] },
  { name: 'Paula Jenkins', skills: [ 'JavaScript', 'Python' ] },
  { name: 'Barbara Jones', skills: [ 'Python', 'Research' ] },
  { name: 'Tracey Young', skills: [ 'Python', 'AutoCAD' ] },
  { name: 'Elizabeth Reed', skills: [ 'Java', 'Python' ] },
  { name: 'Brian Russell', skills: [ 'Python', 'Research' ] },
  { name: 'David Rivera', skills: [ 'Python', 'JavaScript' ] },
  { name: 'Taylor Webb', skills: [ 'Linux', 'Python' ] },
  { name: 'Erin Harris', skills: [ 'AutoCAD', 'Python' ] },
  { name: 'Kyle Lee', skills: [ 'Python', 'JavaScript' ] }
]
mydatabase>
```

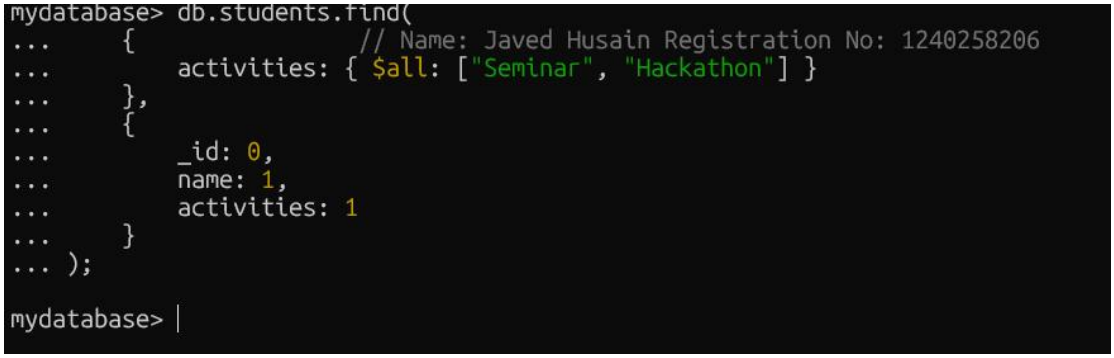
Conclusion :
This query finds students who have "Python" as a skill but do not have "C++" as a skill. The $and operator ensures both conditions are met, while the $ne operator filters out students with "C++". The output displays only the name and skills of the matching students.

Q11. Return names of students who participated in "Seminar" and "Hackathon" both.

Solution :

```
db.students.find(

  {
          // Name: Javed Husain Registration No: 1240258206

    activities: { $all: ["Seminar", "Hackathon"] }
  },

  {
    _id: 0,
    name: 1,
    activities: 1
  }

);
```

Output :



Conclusion :
This query finds all students who have participated in both a "Seminar" and a "Hackathon," and displays their names and activity types. (No record found)

6. Subdocuments and Nested Conditions

Q12. Find students who scored more than 80 in "**Web Development**" **only if** they belong to the "**Computer Science**" department.

Solution :

```
db.enrollments.find(
  {      // Name: Javed Husain Registration No: 1240258206
    course_title: "Web Development",
    marks: { $gt: 80 },
    department: "Computer Science"
  },
  {
    _id: 0,
    student_id: 1,
    marks: 1,
    course_title: 1,
    department: 1
  }
);
```

Output :

Conclusion :

This query is intended to find students who are enrolled in the "Web Development" course, belong to the "Computer Science" department, and have scored more than 80 marks. The find() method is used to apply these nested conditions and project specific fields, such as student_id, marks, course_title, and department, in the output. (No record found).

7. Advanced Aggregation (Challenge Level)

Q13. For each faculty member, list the names of all students enrolled in their courses along with average marks per student per faculty

Solution :

```
db.faculty.aggregate([
        // Name: Javed Husain Registration No: 1240258206
  { $lookup: { from: "courses", localField: "courses", foreignField: "_id", as: "courseInfo" } },
  { $unwind: "$courseInfo" },
  { $lookup: { from: "enrollments", localField: "courseInfo._id", foreignField: "course_id", as: "enrolledStudents" } },
  { $unwind: "$enrolledStudents" },
  { $lookup: { from: "students", localField: "enrolledStudents.student_id", foreignField: "_id", as: "studentInfo" } },
  { $project: { facultyName: "$name", studentName: { $arrayElemAt: ["$studentInfo.name", 0] }, marks: "$enrolledStudents.marks" } },
  { $group: { _id: { facultyName: "$facultyName", studentName: "$studentName" }, averageMarks: { $avg: "$marks" } } },
  { $project: { _id: 0, facultyName: "$_id.facultyName", studentName: "$_id.studentName", averageMarks: 1 } },
  { $sort: { facultyName: 1, studentName: 1 } }
]);
```

Output :

```
mydatabase> db.faculty.aggregate([
...     // Name: Javed Husain Registration No: 1240258206
...     { $lookup: { from: "courses", localField: "courses", foreignField: "_id", as: "courseInfo" } },
...     { $unwind: "$courseInfo" },
...     { $lookup: { from: "enrollments", localField: "courseInfo._id", foreignField: "course_id", as: "enrolledStudents" } },
...     { $unwind: "$enrolledStudents" },
...     { $lookup: { from: "students", localField: "enrolledStudents.student_id", foreignField: "_id", as: "studentInfo" } },
...     { $project: { facultyName: "$name", studentName: { $arrayElemAt: ["$studentInfo.name", 0] }, marks: "$enrolledStudents.marks" } },
...     { $group: { _id: { facultyName: "$facultyName", studentName: "$studentName" }, averageMarks: { $avg: "$marks" } } },
...     { $project: { _id: 0, facultyName: "$_id.facultyName", studentName: "$_id.studentName", averageMarks: 1 } },
...     { $sort: { facultyName: 1, studentName: 1 } }
... ]);
[
  {
    averageMarks: 90,
    facultyName: 'Alexis Stone',
    studentName: 'Anthony Zavala'
  },
  {
    averageMarks: 93,
    facultyName: 'Alexis Stone',
    studentName: 'Barbara Jones'
  },
  {
    averageMarks: 69,
    facultyName: 'Andrew Mcmahon',
    studentName: 'Dr. Michael Griffin Jr.'
  },
  {
    averageMarks: 81,
    facultyName: 'Andrew Mcmahon',
    studentName: 'Megan Taylor'
  },
  {
    averageMarks: 52,
    facultyName: 'Ann Johnson',
    studentName: 'Colleen Todd'
  },
  {
    averageMarks: 59,
    facultyName: 'Ann Porter MD',
    studentName: 'Benjamin White'
```

Conclusion :

This query shows which department has the highest number of students by first counting students in each department, then sorting the results, and finally limiting the output to the top one.

Q14. Show the most popular activity type (e.g., Hackathon, Seminar, etc.) by number of student participants.

Solution :

```
db.students.aggregate([
        // Name: Javed Husain Registration No: 1240258206
  { $unwind: "$activities" },
  { $group: { _id: "$activities", totalParticipants: { $sum: 1 } } },
  { $sort: { totalParticipants: -1 } },
  { $limit: 1 },
  { $project: { _id: 0, activity: "$_id", totalParticipants: 1 } }
]);
```

Output :

Conclusion :
This query uses an aggregation pipeline to find the most popular activity type based on student participation. It unwinds the activities array, groups and counts participants for each activity, sorts the results, and returns only the top result with a clean output.(No record found).